

Joint Latency-Energy Serverless Orchestration in Integrated 6G-Satellite Vehicular Networks

Shahrokh Vahabi¹ and Roberto Riggio²

¹CNIT, Consorzio Nazionale Interuniversitario per le Telecomunicazioni, Parma, Italy

²Università Politecnica delle Marche, Ancona, Italy

Abstract—Future 6G networks aim to deliver global connectivity by integrating terrestrial RoadSide Units (RSUs) with non-terrestrial platforms such as Low Earth Orbit (LEO) satellites. In this environment, serverless Multi-access Edge Computing (MEC) provides a flexible framework for deploying vehicle-to-everything (V2X) applications. However, dynamic task offloading faces significant challenges due to high vehicle mobility, satellite movement, Doppler-induced handovers, serverless container cold starts, and strict energy constraints. To address these issues, this paper proposes a joint latency-energy-aware serverless orchestration framework for integrated 6G satellite-assisted V2X networks. The multi-tier task scheduling problem is formulated as a stochastic optimization model spanning local vehicles, co-located and neighboring RSUs, and LEO satellites. An online scheduler based on Lyapunov Drift-Plus-Penalty (DPP) control dynamically allocates computational tasks to balance latency and energy consumption without requiring prior workload statistics. To further reduce service delays, the framework incorporates proximity-aware proactive container pre-warming and velocity-adaptive task queue deadlines. Experimental results show that the proposed orchestrator maintains system stability while limiting task drops to $\sim 7\%$. Under optimal settings, it achieves an average end-to-end delay of ~ 269 ms and total energy consumption of ~ 3 KJ, outperforming greedy baselines by up to $\sim 32\%$ in latency and $\sim 43\%$ in energy efficiency.

Index Terms—Vehicular edge computing, serverless computing, cold start, TN/NTN networks.

I. INTRODUCTION

The integration of vehicular environments with non-terrestrial satellite networks and terrestrial infrastructures has created a highly dynamic computing landscape in which computation is increasingly bound to mobile physical entities [1]. In this setting, vehicle onboard units, roadside edge servers, and Low Earth Orbit (LEO) satellites collectively form a multi-tier computing fabric capable of supporting computation intensive vehicular applications [2]. Efficient deployment across such heterogeneous infrastructure requires an execution model that can adapt rapidly to mobility and workload variability. Serverless computing, and in particular Function as a Service (FaaS) frameworks, has emerged as a strong candidate because it decouples application logic from persistent resource provisioning [3]. In this model, applications are decomposed into short-lived, event-driven functions that execute only upon demand, aligning well with the sporadic and location dependent nature of vehicular workloads such as cooperative perception and real-time inference, while reducing idle resource consumption [4].

Conventional edge orchestration systems typically rely on virtual machines or containerized services that remain continuously active to ensure low-latency execution. Although this design can provide predictable response times, it is inefficient in vehicular scenarios where task arrivals are intermittent and highly localized. FaaS paradigms instead enable on demand instantiation of lightweight execution environments, reducing baseline resource usage. However, this introduces cold start latency, arising from container image retrieval, initialization, and runtime bootstrapping [5]. In highly mobile environments, such delays can exceed the available communication window between a vehicle and an edge node, making latency sensitive applications vulnerable to service interruption. This motivates the need for scheduling mechanisms that jointly consider mobility patterns and runtime initialization delays [6].

The expansion of non-terrestrial networks further extends this computing continuum by transforming LEO satellites from passive relays into active computational nodes. Modern satellite constellations can execute containerized workloads in orbit and provide coverage in regions where terrestrial infrastructure is unavailable or congested [7]. However, the inherent orbital motion of satellites induces a continuously evolving network topology [8]. Connectivity between vehicles and satellites is constrained by short visibility windows and time varying channel conditions, including propagation delay variability and Doppler-induced frequency shifts [9]. Effective orchestration in such environments requires joint awareness of both terrestrial edge resource states and satellite orbital dynamics, replacing static routing strategies with adaptive decision-making mechanisms.

This work addresses these challenges by proposing a measurement-driven online serverless orchestration framework based on Lyapunov drift plus penalty optimization. The proposed scheduler dynamically assigns vehicular tasks to appropriate computing tiers by jointly minimizing latency and energy consumption without requiring prior statistical knowledge of workload distributions. Instead of relying on heavyweight predictive models, it leverages real-time mobility information to enable proximity-aware container pre-warming at forthcoming roadside nodes, thereby reducing cold start penalties. In addition, velocity adaptive task deadlines are introduced to adjust queueing constraints according to vehicle speed, preventing backlog accumulation and reducing service of obsolete tasks.

The remainder of this paper is organized as follows. Section II reviews related work, Section III introduces the proposed methodology, and Section IV details the testbed setup. Section V presents the results, and Section VI concludes the paper and outlines future research directions.

II. RELATED WORK

A. Strategies for Energy Efficient Offloading

Hu et al. proposed a scheduling approach that addresses the high power demands of edge computing servers [10]. They created a system that intelligently switches servers between active and sleep modes based on the deadlines of incoming applications. By dynamically managing these power states, the model significantly reduces the electricity consumption required to maintain vehicular network operations. To achieve this objective, the authors employed a decentralized auction-based strategy that evaluates tasks modeled as directed acyclic graphs and prioritizes their execution order. This mechanism enables critical tasks to be promoted within the scheduling queue without disrupting previously scheduled processes.

Bayesian optimization techniques were employed by Lv et al. to address power consumption in partial task offloading scenarios [11]. Their framework minimizes the total energy expenditure associated with local computation, data transmission, and edge processing. Quality of service requirements are treated as strict constraints, ensuring efficient resource allocation without compromising service guarantees. To navigate the large optimization space, the authors introduced a specialized tuning mechanism that guides the search process toward near-optimal solutions while reducing computational complexity.

Moghaddasi et al. explored the application of double deep Q-networks for data management in modern cellular environments [12]. Their reinforcement learning agent adapts to dynamic traffic conditions, varying vehicle speeds, and fluctuating signal strengths. Experimental results demonstrated a reduction in communication overhead and an improvement in overall energy efficiency. To stabilize the learning process, the framework incorporates an experience replay mechanism that samples historical network transitions and continuously refines decision-making performance.

To alleviate congestion in local data centers during peak traffic periods, Yadav et al. developed a three-phase heuristic offloading scheme [13]. The proposed approach transfers computational workloads from overloaded network nodes to nearby vehicles with available resources. By balancing energy conservation and processing performance, the framework promotes sustainable operational costs. The process involves identifying overloaded servers, selecting suitable tasks for migration, and allocating resources based on dynamically estimated vehicle dwell times.

B. Scheduling for Latency Efficiency

Feng et al. introduced a reverse offloading paradigm to mitigate network bottlenecks [15]. Instead of forwarding all computational tasks to edge servers, their framework identifies

overloaded RoadSide Units (RSUs) and redirects resource-intensive tasks to capable vehicles. This alternative task distribution strategy effectively minimizes maximum processing latency across the network. To support partitioned task execution, the authors designed a joint alternating-optimization and bisection-search algorithm that determines the optimal workload allocation between edge nodes and vehicles.

Digital twin technology was leveraged by Jeremiah et al. to reduce task completion delays through virtual replication of the physical network [16]. The framework simulates resource allocation decisions before implementing them in the real environment. By encouraging cooperation among neighboring edge nodes, the proposed approach improves computational throughput and reduces response times. Furthermore, non-orthogonal multiple access technology was integrated to enhance spectral efficiency and network capacity.

Cao et al. also utilized digital twin technology to address the complex dynamics of vehicular mobility [17]. Their framework considers five optimization objectives, with particular emphasis on reducing downlink latency and computational delay. An evolutionary optimization algorithm adapts to varying vehicle speeds to ensure efficient delivery of processed data. To address the complexity of multi-objective optimization, the authors introduced a variable-grouping strategy that decomposes the problem into manageable subproblems.

Combining task offloading with intelligent caching, Xue et al. proposed a reinforcement learning framework for latency reduction in vehicular edge networks [18]. The system proactively predicts and stores required service programs at edge nodes before requests are generated. As a result, application requests can be processed with minimal delay, avoiding retrieval from distant cloud servers. Coordination among cloud servers, edge nodes, and vehicular processors is achieved through a deep deterministic policy gradient algorithm.

C. Gap Analysis

This paper proposes a unified, measurement-driven serverless orchestration framework for terrestrial satellite vehicular networks, bridging the gap between theoretical optimization and practical constraints. Rather than relying on heavy prediction models, our approach uses real-time vehicle mobility coordinates to trigger proximity aware proactive container pre-warming at upcoming roadside gateways, minimizing cold start latencies. Additionally, the framework implements velocity adaptive task deadlines to prevent queue congestion under dynamic vehicular speeds. We evaluate the proposed architecture on a hybrid testbed combining microtraffic simulations, live satellite orbital trajectories, and physical containerized edge deployments, demonstrating the practical feasibility and performance gains of joint latency-energy optimization.

III. PROPOSED JOINT LATENCY-ENERGY SERVERLESS ORCHESTRATION FRAMEWORK

A. System Model

We consider a dynamic urban vehicular network with three tiers: vehicles, terrestrial edge infrastructure, and LEO

satellites. The vehicular tier comprises a set of vehicles $\mathcal{V} = \{1, \dots, N_V\}$, each equipped with an onboard unit for local execution or V2V offloading to neighboring vehicles. The terrestrial tier consists of RSUs $\mathcal{R} = \{1, \dots, N_R\}$, each co-located with an edge server and interconnected through a high-speed metropolitan xHaul network. The space tier comprises LEO satellites $\mathcal{S} = \{1, \dots, N_S\}$, which provide computing and backhaul services when terrestrial resources are congested or unavailable. Vehicles generate requests for serverless functions from the set $\mathcal{F}$. At each time slot t , a request can be assigned to an active execution target in $\mathcal{N}(t) = \{\text{V2V}, \text{RSU}, \text{LEO}\}$, representing local or neighboring-vehicle execution, co-located or adjacent RSU execution, and satellite execution, respectively. Each candidate node $j \in \mathcal{N}(t)$ maintains a task queue $Q_j(t)$ containing its unprocessed requests.

B. Latency Factor

The end-to-end latency of function $f \in \mathcal{F}$ assigned to node $j \in \mathcal{N}(t)$ at time slot t comprises network and processing delays:

$$L_j^f(t) = L_j^{\text{net},f}(t) + L_j^{\text{proc},f}(t) \quad (1)$$

The network latency is:

$$L_j^{\text{net},f}(t) = D_j^{\text{trans}}(t) + D_j^{\text{prop}}(t) + D_j^{\text{route}}(t) + D_j^{\text{ho}}(t) \quad (2)$$

where $D_j^{\text{trans}}(t) = D_f^{\text{data}}/C_j(t)$ is the transmission delay, with D_f^{data} denoting the input data size of function f and $C_j(t)$ denoting the transmission capacity between the requesting vehicle and node j . Furthermore, $D_j^{\text{prop}}(t)$ is the signal propagation delay, $D_j^{\text{route}}(t)$ is the waiting time in intermediate gateway and switch buffers, and $D_j^{\text{ho}}(t)$ is the connection handover signaling delay. Propagation delay is negligible for local execution, small for terrestrial V2V and V2I links, and substantially higher for satellite links because of their longer communication distance. Terrestrial handover delay is modeled as a constant, whereas satellite handover delay depends on the Doppler shift induced by the relative motion of vehicle i and satellite k . Let f_c denote the carrier frequency, c the speed of light, $\mathbf{v}_i(t)$ and $\mathbf{v}_k^{\text{sat}}(t)$ their velocity vectors, and $\mathbf{r}_{i,k}(t)$ their relative position vector. The Doppler shift is

$$\Delta f_{\text{Dop}}(t) = f_c \frac{(\mathbf{v}_i(t) - \mathbf{v}_k^{\text{sat}}(t)) \cdot \mathbf{r}_{i,k}(t)}{c \|\mathbf{r}_{i,k}(t)\|} \quad (3)$$

Because Doppler compensation increases synchronization, connection setup, and signaling overhead, the satellite handover delay is modeled as:

$$D_{\text{sat}}^{\text{ho}}(t) = D_0^{\text{ho}} (1 + \mu |\Delta f_{\text{Dop}}(t)|) \quad (4)$$

where $\mu \geq 0$ is the Doppler-sensitivity coefficient, configured as $\mu = 10^{-4}$ in the testbed [29]. The processing latency comprises execution, cold-start, and queueing delays:

$$L_j^{\text{proc},f}(t) = T_{j,f}^{\text{exec}}(t) + T_{j,f}^{\text{cold}}(t) + D_{j,f}^{\text{exec},q}(t) \quad (5)$$

where $T_{j,f}^{\text{exec}}(t)$ is the execution time of function f on node j , $T_{j,f}^{\text{cold}}(t)$ is the container initialization delay, which is zero when the container is warm, and $D_{j,f}^{\text{exec},q}(t)$ is the waiting time in the execution queue of node j .

C. Queue Dynamics and Drop Policy

We define the task queue backlog $Q_j(t)$ of node j as:

$$Q_j(t+1) = \max[0, Q_j(t) - \mu_j(t)] + A_j(t) \quad (6)$$

where $A_j(t)$ is the arrival rate of tasks routed to node j , and $\mu_j(t)$ is the processing service rate of node j . Requests are dropped due to two physical reasons, namely queue buffer overflows and queue timeouts. Because the vehicular channel state changes rapidly at high-speeds, control and telemetry packets quickly become stale and invalid. Thus, instead of a static timeout threshold, we define a velocity adaptive timeout threshold $T_f^{\text{timeout}}(t)$ that is inversely proportional to the speed $v_i(t)$ of vehicle i :

$$T_f^{\text{timeout}}(t) = \frac{\theta_f}{1 + \gamma_f \cdot v_i(t)} \quad (7)$$

where θ_f is the baseline timeout threshold when stationary, and $\gamma_f \geq 0$ is a speed sensitivity coefficient and it is configured as $\gamma_f = 0.01$ in our testbed for all functions to dynamically adjust the timeout threshold based on the vehicle speed [20].

D. Energy and Bandwidth Models

The energy footprint of scheduling decision j for function f at slot t is represented by the variable $E_j^f(t)$ as:

$$E_j^f(t) = E_j^{\text{trans}}(t) + E_{j,f}^{\text{comp}}(t) + E_{j,f}^{\text{cold}}(t) \quad (8)$$

The transmission energy, $E_j^{\text{trans}}(t)$, is the energy consumed by the transmitter, calculated as the product of the transmitter power variable P_j^{trans} and the transmission delay. The computation energy, $E_{j,f}^{\text{comp}}(t)$, is the energy consumed by the edge node while running the function, calculated as the product of the computation power variable P_j^{comp} of the node and the execution run time. The container startup energy, $E_{j,f}^{\text{cold}}(t)$, represents the transient energy spike required to boot a cold container at the edge node. The satellite bandwidth cost represents the resource utilization cost of using space-based satellite backhaul links. This cost is represented by the variable $B_j(t) = B_{\text{sat}}$ if the request is scheduled to the satellite, and $B_j(t) = 0$ if the request is scheduled to terrestrial edge nodes. To model communication capacity across the different tiers, the channel transmission capacity is formulated using the Shannon capacity equation as [21]:

$$C_j(t) = B_j \log_2(1 + \text{SINR}_j(t)) \quad (9)$$

where B_j represents the channel bandwidth allocated for scheduling option j . Specifically, the bandwidth is configured as $B_{\text{V2I}} = 100$ MHz for V2I links, $B_{\text{V2V}} = 20$ MHz for V2V sidelinks, and $B_{\text{sat}} = 50$ MHz for satellite links [22]–[24]. Terrestrial links use log distance path loss models to calculate the Signal to Interference plus Noise Ratio $\text{SINR}_j(t)$, while the satellite uplink uses free space path loss at a 20 GHz carrier frequency [20], [21]. Local onboard execution uses a constant capacity of 1 Gbps [25].

E. Online Lyapunov Optimization

To solve this multi-objective optimization problem dynamically without requiring future channel states, task arrivals, or energy dynamics, we employ the Online Lyapunov Drift Plus Penalty (DPP) framework. Our objective is to minimize both the time average energy consumption and average execution latency:

$$\begin{aligned} \min \quad & \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E} [\omega_E E_{j(t)}^f(t) + \omega_L L_{j(t)}^f(t)] \\ \text{s.t.} \quad & \lim_{T \rightarrow \infty} \frac{1}{T} \mathbb{E} [Q_j(t)] = 0, \quad \forall j \end{aligned} \quad (10)$$

where $\omega_E \geq 0$ and $\omega_L \geq 0$ are the two weighting coefficients for energy and latency, respectively. We define a quadratic Lyapunov function, $L(\mathbf{Q}(t))$, representing the physical backlog state of the system:

$$L(\mathbf{Q}(t)) = \frac{1}{2} \sum_{j \in \mathcal{N}} (Q_j(t))^2 \quad (11)$$

where $Q_j(t)$ represents the physical task queue length at execution node j . The one step Lyapunov drift is defined as the expected change in the Lyapunov function over a single time slot:

$$\Delta(\mathbf{Q}(t)) = \mathbb{E}[L(\mathbf{Q}(t+1)) - L(\mathbf{Q}(t)) \mid \mathbf{Q}(t)] \quad (12)$$

To stabilize the physical queues while minimizing the joint latency-energy objective, we minimize the Lyapunov DPP as:

$$\Delta(\mathbf{Q}(t)) + V_{\text{ctrl}} \cdot \mathbb{E}[\omega_E E_{j(t)}^f(t) + \omega_L L_{j(t)}^f(t) \mid \mathbf{Q}(t)] \quad (13)$$

where $V_{\text{ctrl}} > 0$ is a control parameter that scales the priority of joint objective optimization relative to physical queue stabilization. To balance the trade-off between energy and latency, the coefficients ω_E and ω_L are initialized based on dimensional normalization as:

$$\omega_E = \frac{\alpha}{E^{\max}}, \quad \omega_L = \frac{1 - \alpha}{L^{\max}} \quad (14)$$

where $E^{\max}$ and $L^{\max}$ represent the maximum expected energy footprint and end-to-end latency per task execution, respectively, mapping the metrics into a dimensionless domain. The parameter $\alpha \in [0, 1]$ is a relative weight that can be statically initialized or dynamically adjusted by the system operator to prioritize resource management. Setting $\alpha = 0$ yields a pure latency minimization scheme, whereas $\alpha = 1$ forces the system to operate as a green, energy minimizing scheduler. Intermediate values of the parameter provide a balanced trade-off, with the specific values chosen empirically based on the vehicle battery level and real-time channel conditions. By bounding the drift and applying the opportunistic minimization framework, the orchestrator makes a scheduling decision $j_k^*(t)$ for each incoming request k of function f generated at time slot t . In a multi-vehicle and multi-task environment, multiple task requests can be generated simultaneously in a single time step. For each individual task request, a separate

optimal offloading target is computed by solving the following deterministic minimization problem at every time slot:

$$j_k^*(t) = \arg \min_{j \in \mathcal{N}(t)} \left[Q_j(t) \cdot L_j^f(t) + V_{\text{ctrl}} \cdot (\omega_E E_j^f(t) + \omega_L L_j^f(t)) \right] \quad (15)$$

In this scheduling rule, the first term acts as a queue stability and congestion control penalty. The term $Q_j(t)$ represents the physical queue backlog at execution node j . The second term represents the joint optimization penalty. Under the DPP framework, the time average backlog across the physical queues is bounded by $O(V_{\text{ctrl}})$, implying that the queuing latency scales linearly with V_{ctrl} . Simultaneously, the time average optimization cost approaches the global offline optimal limit at a rate of $O(1/V_{\text{ctrl}})$. A key advantage of this formulation is that it is solver-free; since the candidate destination set is discrete and small, the optimal routing decision is resolved in microseconds via a simple cost comparison.

IV. TESTBED SETUP

To validate the framework, we generate a one-hour mobility trace of Ancona, Italy, using SUMO 1.18.0 and TraCI. The scenario includes 80 vehicles, of which 25% have onboard microservers, and 12 RSUs. The trace is imported into ns-3.27 to model terrestrial and satellite links and route packets through TAP interfaces to containerized faasd¹ edge instances. V2I links are modeled using the 3GPP NR 5G LENA module in ns-3.27², operating at 3.6 GHz with 100 MHz bandwidth [22], OFDMA scheduling [29], 2×4 UE antennas, and 8×8 gNodeB antennas. V2V sidelinks use 20 MHz bandwidth and log-distance path loss. The space tier comprises eight LEO satellites at an altitude of 600 km [27] and a velocity of 7.5 km/s [20], using a 20 GHz Ka-band uplink with slant-range path loss [28]. Satellite trajectories are propagated using live CelesTrak³ data, and the highest-elevation satellite is selected at each time slot. The resulting Doppler shift dynamically scales a 40 ms baseline delay [29]. The emulation runs on a Rocky Linux 10.1 virtual machine with 12 vCPUs, an Intel Xeon Gold 6252N CPU, and 64 GB of RAM. The workload contains five serverless functions: image processing (100 KB), LiDAR processing (500 KB), video analytics (1000 KB), path planning (10 KB), and sensor fusion (150 KB). We set the queue capacity to $Q^{\max} = 50$ tasks and the Lyapunov weight to $\alpha = 0.4$, while containers run on a single node k3s 1.28 cluster. End-to-end latency is measured through HTTP POST round-trip times, and power consumption is monitored using Scaphandre⁴ and Prometheus⁵. A sample one-hour LEO dataset is publicly available at GitHub repo⁶ [30].

¹<https://github.com/openfaas/faasd>

²<https://github.com/QiuYukang/5G-LENA/tree/master>

³<https://celestrak.org/NORAD/elements/>

⁴<https://github.com/hubblo-org/scaphandre>

⁵<https://github.com/prometheus/prometheus>

⁶https://github.com/shahrokhvhabhi/NexaSphere/tree/main/realworld/leo_orbit

V. EXPERIMENTAL RESULTS

We evaluate the proposed Lyapunov DPP framework for $V_{\text{ctrl}} \in \{1, 10, 50, 100, 300, 500\}$ against three baselines. Latency-Greedy selects the node with the lowest immediate end-to-end latency but ignores queue backlog and energy consumption. Energy-Greedy minimizes the immediate energy consumption of transmission, computation, and container cold starts without considering queueing delays or timeouts. Least-Loaded selects the node with the smallest queue backlog while ignoring network conditions and container states [5].

A. Average Latency

As illustrated in Fig. 1, the proposed Lyapunov framework reaches its minimum average end-to-end latency of ~ 269 ms at $V_{\text{ctrl}} = 100$, outperforming the Latency-Greedy by ~ 310 ms, Least-Loaded by ~ 355 ms, and Energy-Greedy ~ 395 ms baselines.

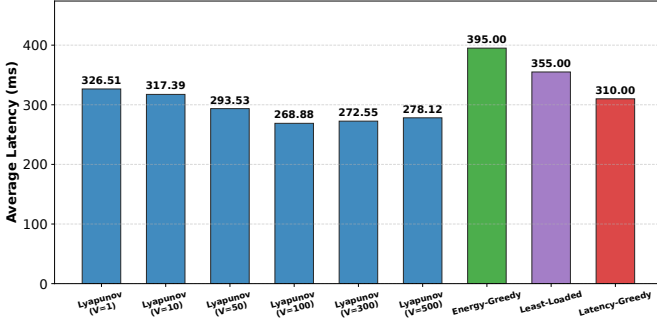


Fig. 1. Average latency across different policies and Lyapunov control parameter V_{ctrl} values.

Under the proposed framework, routing is driven by the dynamic minimization of Expression (15), where the term $Q_j(t) \cdot L_j^f(t)$ acts as a congestion tax scaling with the queue backlog at target j . At low control parameters such as $V_{\text{ctrl}} = 1$ and $V_{\text{ctrl}} = 10$, average latency remains high at ~ 327 ms and ~ 317 ms because the scheduler prioritizes queue stabilization over execution speed. Increasing V_{ctrl} to 100 shifts priority toward penalty minimization, enabling the scheduler to select high-performance edge servers and warm container environments, which minimizes cold starts and lowers latency to about 269 ms. However, at $V_{\text{ctrl}} = 300$ and $V_{\text{ctrl}} = 500$, latency rises to ~ 273 ms and ~ 278 ms. The Latency-Greedy policy by ~ 310 ms, ignores the backlog tax $Q_j(t)$, routing tasks to the closest RSUs and causing severe queue overflows. The Least-Loaded policy by ~ 355 ms, optimizes queue balance but ignores network link quality, routing requests over distant or low bandwidth links. Finally, the Energy-Greedy policy by ~ 395 ms, forces local execution on slow onboard processors or selects low power nodes regardless of queue status, resulting in buffer congestion and high task drop rates.

B. Container Cold Starts

Container cold starts represent occurrences where a function must be initialized from scratch at an edge server, introducing

a transient boot delay and a significant energy spike. As illustrated in Fig. 2, the proposed Lyapunov framework with $V_{\text{ctrl}} = 100$ records its lowest cold start count of 130. This represents relative reductions of 10.34% compared to the Latency-Greedy baseline of 145 cold starts, 19.75% compared to the Least-Loaded baseline of 162 cold starts, and 27.78% compared to the Energy-Greedy baseline of 180 cold starts. Lower values like $V_{\text{ctrl}} = 1$ and $V_{\text{ctrl}} = 10$ prioritize queue

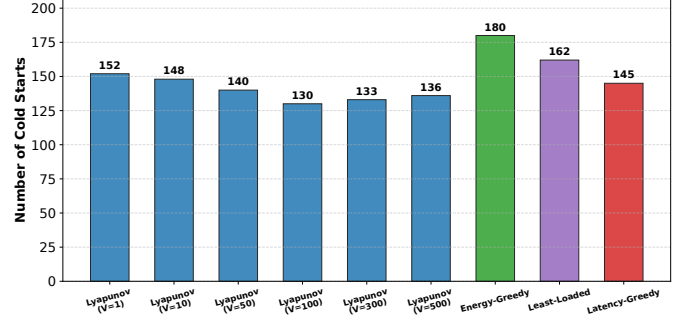


Fig. 2. Cold start counts across different policies and Lyapunov control parameter V_{ctrl} values.

stability, forcing the scheduler to distribute tasks across idle edge servers and triggering 152 and 148 cold starts. Increasing V_{ctrl} to 100 prioritizes the joint Latency-Energy penalty, routing tasks to warm containers and reducing cold starts to 130. At higher settings like $V_{\text{ctrl}} = 300$ and $V_{\text{ctrl}} = 500$, cold starts rise to 133 and 136 because accumulated queue backlogs eventually force overflow routing to cold servers.

C. Handover Rate

The handover rate measures the percentage of offloaded tasks requiring active signaling from horizontal routing to adjacent RSUs via xHaul networks and vertical handovers to satellites. As shown in Fig. 3, the Lyapunov framework maintains a balanced handover rate ranging from 12.50% at $V_{\text{ctrl}} = 1$ to a peak of 15.40% at $V_{\text{ctrl}} = 100$. The Lyapunov scheduler at $V_{\text{ctrl}} = 100$ reduces the handover rate by 18.80% compared to the Least-Loaded baseline which has a rate of 34.20%. This represents a relative reduction of $\sim 55\%$ in signaling overhead.

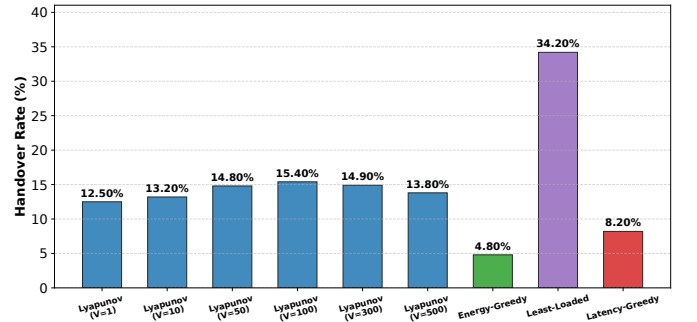


Fig. 3. Handover rate across different policies and Lyapunov control parameter V_{ctrl} values.

By contrast, the Latency-Greedy policy and the Energy-Greedy policy exhibit lower handover rates of 8.20% and 4.80%. These low rates minimize signaling but cause severe performance degradation. As the control parameter V_{ctrl} in the Lyapunov scheduler increases from 1 to 100, the handover rate rises from 12.50% to 15.40% because the optimization framework places a higher priority on minimizing the joint Latency-Energy penalty, utilizing adjacent RSUs and satellite links to avoid queuing delays. However, as V_{ctrl} increases further to 300 and 500, the handover rate decreases to 14.90% and 13.80% since relaxed queue constraints allow the scheduler to buffer more tasks in local and co-located queues.

D. Drop Rate

As presented in Table I, the proposed Lyapunov framework with $V_{ctrl} = 100$ achieves the lowest task drop rate of 6.61%. This performance represents a significant improvement, reducing the drop rate by 20.68% compared to the Least-Loaded baseline rate of 27.29%, by 27.53% compared to the Latency-Greedy baseline rate of 34.14%, and by 28.19% compared to the Energy-Greedy baseline rate of 34.80%.

TABLE I
TASK DROP RATE COMPARISON

Policy	Drop Rate (%)
Lyapunov ($V_{ctrl} = 100$)	6.61
Lyapunov ($V_{ctrl} = 300$)	6.71
Lyapunov ($V_{ctrl} = 50$)	7.16
Lyapunov ($V_{ctrl} = 500$)	7.28
Lyapunov ($V_{ctrl} = 10$)	8.59
Lyapunov ($V_{ctrl} = 1$)	9.04
Least-Loaded	27.29
Latency-Greedy	34.14
Energy-Greedy	34.80

At a low parameter of $V_{ctrl} = 1$, the drop rate is 9.04% because the scheduler prioritizes queue stability, routing tasks to less congested but slower satellite links with high propagation and handover delays, which triggers timeouts. The optimal parameter of $V_{ctrl} = 100$ minimizes drops to 6.61%. At a high parameter of $V_{ctrl} = 500$, the drop rate rises back to 7.28% because relaxed queue stabilization constraints permit larger backlog accumulations that lead to longer waiting times in the buffers, directly triggering timeouts.

E. Latency-Energy Trade-offs

As illustrated in Fig. 4, both the average end-to-end latency and the average task energy consumption follow a matching convex trajectory, reaching their lowest values simultaneously at $V_{ctrl} = 100$. Specifically, the average task energy consumption drops from ~ 0.16 J at $V_{ctrl} = 1$ to a minimum of ~ 0.12 J at $V_{ctrl} = 100$, representing a relative reduction of 21.43% in the per task energy footprint, while the average per task latency decreases by 17.65% from ~ 327 ms to ~ 269 ms. At low values like $V_{ctrl} = 1$ and $V_{ctrl} = 10$, the optimization process is dominated by queue stabilization, forcing the orchestrator to offload tasks to idle but computationally slower edge servers or distant satellites.

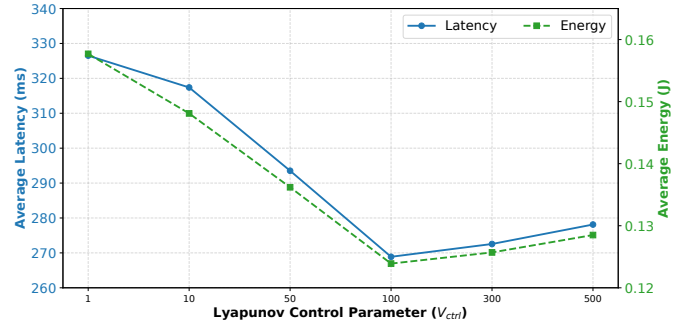


Fig. 4. Lyapunov control parameter V_{ctrl} sweep showing the dual-axis trade-off between average end-to-end latency (ms) and average energy consumption (J) per task.

Scaling V_{ctrl} to 100 gives the orchestrator flexibility to tolerate minor queue backlogs, prioritizing the minimization of the joint latency-energy penalty and routing tasks to high-capacity RSUs with warm containers. However, at higher parameters of $V_{ctrl} = 300$ and $V_{ctrl} = 500$, relaxed constraints allow backlogs to accumulate in the buffers, directly inflating queuing delays.

F. Total Energy Consumption

As shown in Fig. 5, the proposed scheduler achieves its minimum total energy consumption of 2.75 KJ at $V_{ctrl} = 100$, saving 17.91%, 31.25%, and 42.71% over Energy-Greedy, Least-Loaded, and Latency-Greedy, respectively.

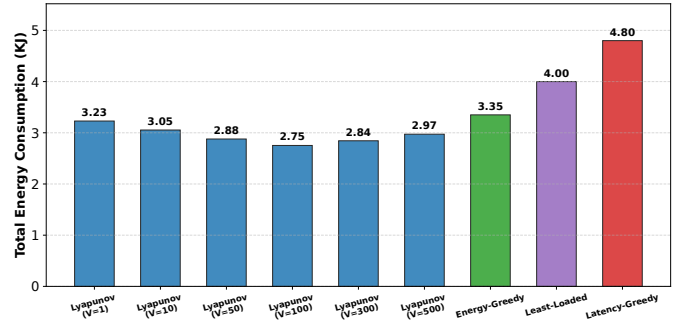


Fig. 5. Total energy consumption across different scheduling policies and Lyapunov control parameter V_{ctrl} values.

Although Energy-Greedy minimizes the immediate cost of each decision, it ignores the longer-term effects of queue congestion, container reuse, and task expiration. It consequently causes 180 container cold starts and a task drop rate of approximately 35%, allocating resources to requests that eventually time out. In contrast, the proposed scheduler jointly considers queue backlog, latency, and energy, routing requests to warm and uncongested instances. At $V_{ctrl} = 100$, it reduces cold starts to 130 and limits the task drop rate to 6.61%. Latency-Greedy consumes the most energy (4.80 KJ) because it frequently selects high-power options, while Least-Loaded consumes 4.00 KJ by routing requests without considering link or container conditions. The Lyapunov configurations exhibit a convex trend: strict queue stabilization results in

3.23 KJ at $V_{\text{ctrl}} = 1$, whereas relaxed queue control causes backlog accumulation and overflow routing, increasing consumption to 2.97 KJ at $V_{\text{ctrl}} = 500$.

VI. CONCLUSION AND FUTURE WORK

This paper presented a unified online serverless orchestration framework for terrestrial satellite V2X networks, designed to dynamically optimize execution latency and energy consumption. By implementing a solver-free Lyapunov scheduler with proximity aware container pre-warming and velocity adaptive timeouts, the system avoids heavy prediction models and computational overheads. Evaluated on a hybrid hardware in the loop testbed, the proposed scheduler operating at $V_{\text{ctrl}} = 100$ achieves a minimum average latency of ~ 269 ms, total energy consumption of 2.75 KJ, and task drop rate of $\sim 7\%$. Also, the proposed scheduler reduces average latency by $\sim 13 - 32\%$ and total energy consumption by $\sim 18 - 43\%$ compared to the baseline policies. Future research will focus on extending this framework with more intelligent approaches to train the model to optimize parameters more dynamically using varying channel quality, traffic density, and edge server loads.

ACKNOWLEDGMENT

This work was funded by the European Union under the Horizon Europe programme through the SNS JU (Grant Agreement No. 101192912, NexaSphere). Views expressed are those of the authors and do not necessarily reflect those of the EU or the SNS JU.

REFERENCES

- [1] G. Geraci, D. López-Pérez, M. Benzaghta *et al.*, “Integrating terrestrial and non-terrestrial networks: 3d opportunities and challenges,” *IEEE Communications Magazine*, vol. 61, no. 4, pp. 42–48, 2023.
- [2] F. D. Rossi, P. S. S. de Souza, and M. C. Luizelli, “Resource allocation on low-earth orbit edge infrastructure: Taxonomy, survey, and research challenges,” *IEEE Access*, vol. 13, pp. 40 992–41 010, 2025.
- [3] S. Vahabi, F. Righetti, C. Vallati *et al.*, “The impact of prediction models on energy-aware resource management in faas platforms,” *IEEE Access*, vol. 13, pp. 85 711–85 727, 2025.
- [4] S. Vahabi, F. Righetti, C. Vallati *et al.*, “Energy-efficient resource management for real-time applications in faas edge computing platforms,” in *Proceedings of the IEEE/ACM International Conference on Utility and Cloud Computing*, ser. UCC ’23. Association for Computing Machinery, 2024.
- [5] S. Vahabi, F. Righetti, C. Vallati *et al.*, “A predictive hybrid scheduler to reduce completion times of heterogeneous functions in faas platforms,” in *International Conference on Federated Learning Technologies and Applications (FLTA)*, 2025, pp. 456–462.
- [6] W. Zhang, M. Feng, and M. Krunz, “Latency estimation and computational task offloading in vehicular mobile edge computing applications,” *IEEE Transactions on Vehicular Technology*, vol. 73, no. 4, pp. 5808–5823, 2024.
- [7] C. J. Rojas, F. Patrone, J. A. Fraire *et al.*, “From emerging leo satellite constellations to the space cloud: Emulation platforms and orchestration methods,” *Computer Networks*, vol. 276, no. 111970, pp. 1–14, 2026.
- [8] S. Li, Q. Wu, and R. Wang, “Dynamic discrete topology design and routing for satellite-terrestrial integrated networks,” *IEEE/ACM Transactions on Networking*, vol. 32, no. 5, pp. 3840–3853, 2024.
- [9] R. Ortigueira, S. Montejo-Sánchez, S. Henn *et al.*, “Satellite visibility prediction for constrained devices in direct-to-satellite iot systems,” *IEEE Sensors Journal*, vol. 24, no. 16, pp. 26 630–26 644, 2024.
- [10] B. Hu, Y. Shi, and Z. Cao, “Adaptive energy-minimized scheduling of real-time applications in vehicular edge computing,” *IEEE Transactions on Industrial Informatics*, vol. 19, no. 5, pp. 6895–6906, 2023.
- [11] W. Lv, P. Yang, T. Zheng *et al.*, “Energy consumption and qos-aware co-offloading for vehicular edge computing,” *IEEE Internet of Things Journal*, vol. 10, no. 6, pp. 5214–5225, 2023.
- [12] K. Moghaddasi, S. Rajabi, F. S. Gharehchopogh *et al.*, “An energy-efficient data offloading strategy for 5g-enabled vehicular edge computing networks using double deep q-network,” *Wireless Personal Communications*, vol. 133, pp. 2019–2064, 2023.
- [13] R. Yadav, W. Zhang, O. Kaiwartya *et al.*, “Energy-latency tradeoff for dynamic computation offloading in vehicular fog computing,” *IEEE Transactions on Vehicular Technology*, vol. 69, no. 12, pp. 14 198–14 211, 2020.
- [14] P. Li, Z. Xiao, X. Wang *et al.*, “Eptask: Deep reinforcement learning based energy-efficient and priority-aware task scheduling for dynamic vehicular edge computing,” *IEEE Transactions on Intelligent Vehicles*, vol. 9, no. 1, pp. 1830–1846, 2024.
- [15] W. Feng, N. Zhang, S. Li *et al.*, “Latency minimization of reverse offloading in vehicular edge computing,” *IEEE Transactions on Vehicular Technology*, vol. 71, no. 5, pp. 5343–5357, 2022.
- [16] X. Dai, Z. Xiao, H. Jiang *et al.*, “Uav-assisted task offloading in vehicular edge computing networks,” *IEEE Transactions on Mobile Computing*, vol. 150, pp. 243–254, 2023.
- [17] B. Cao, Z. Li, X. Liu *et al.*, “Mobility-aware multiobjective task offloading for vehicular edge computing in digital twin environment,” *IEEE Journal on Selected Areas in Communications*, vol. 41, no. 10, pp. 3046–3055, 2023.
- [18] Z. Xue, C. Liu, C. Liao *et al.*, “Joint service caching and computation offloading scheme based on deep reinforcement learning in vehicular edge computing systems,” *IEEE Transactions on Vehicular Technology*, vol. 72, no. 5, pp. 6709–6722, 2023.
- [19] M. Wang, J. Mao, W. Zhao *et al.*, “Smart city transportation: A vanet edge computing model to minimize latency and delay utilizing 5g network,” *J Grid Computing*, vol. 22, no. 25, pp. 1–13, 2024.
- [20] S. Bhandari and T. X. V. nad Symeon Chatzinotas, “Leo-based edge computing service platform for challenging geographical terrain,” *IEEE Open Journal of the Communications Society*, vol. 6, pp. 9991–10009, 2025.
- [21] M. R. Thadathil and N. Saeed, “Smart traffic intersections: Leveraging isac and millimeter-wave for advanced vehicle platooning,” in *Global Information Infrastructure and Networking Symposium (GIIS)*, 2025, pp. 1–6.
- [22] A. Pfadler, C. Ballesteros, J. Romeu *et al.*, “Hybrid massive mimo for urban v2i: Sub-6 ghz vs mmwave performance assessment,” *IEEE Transactions on Vehicular Technology*, vol. 69, no. 5, pp. 4652–4662, 2020.
- [23] J. Zhang, J. Qiu, X. Lin *et al.*, “Latency analysis and evaluation of c-v2x communications using the 5g uu interface,” *IEEE Internet of Things Journal*, vol. 11, no. 24, pp. 40 506–40 515, 2024.
- [24] A. Alhashemi, R. Samy, A. T. Hafez *et al.*, “Relay satellites for high throughput transmission in earth observation missions,” in *International Conference on Future Telecommunications and Artificial Intelligence (IC-FTAI)*, 2024, pp. 1–6.
- [25] K. Nakano, A. Egami, H. Motozuka *et al.*, “Data collection platform for smart city with gigabit v2x communication over 60 ghz band,” in *IEEE 93rd Vehicular Technology Conference (VTC2021-Spring)*, 2021, pp. 1–5.
- [26] Z. Khan, S. M. Khan, M. Chowdhury *et al.*, “Performance evaluation of 5g millimeter-wave-based vehicular communication for connected vehicles,” *IEEE Access*, vol. 10, pp. 31 031–31 042, 2022.
- [27] Q. Li, J. Xu, W. Li *et al.*, “Variation analysis of atmospheric density at 600km,” in *International Conference on Wireless Communications and Smart Grid (ICWCSG)*, 2021, pp. 204–208.
- [28] R. S. Hao, J. F. Zhang, S. C. Jin *et al.*, “K/ka-band shared-aperture phased array with wide bandwidth and wide beam coverage for leo satellite communication,” *IEEE Transactions on Antennas and Propagation*, vol. 71, no. 1, pp. 672–680, 2023.
- [29] M. Neinavaie and Z. M. Kassas, “Unveiling starlink leo satellite ofdm-like signal structure enabling precise positioning,” *IEEE Transactions on Aerospace and Electronic Systems*, vol. 60, no. 2, pp. 2486–2489, 2024.
- [30] S. Vahabi and R. Riggio, “Leo satellite orbital dataset for urban road networks (ancona, italy),” 2026. [Online]. Available: <https://dx.doi.org/10.21227/1wjs-ds49>